Schema Linker SFT Pipeline

Chia-Han Chen
Department of Computer Science and Engineering
University of California San Diego
La Jolla, CA 92092
chc016@ucsd.edu

Emily Chen
Halıcıoğlu Data Science Institute
University of California San Diego
La Jolla, CA 92092
emc001@ucsd.edu

1 Training Data Methodology

1.1 Composing Training Data

The training data was based on `train.json` that is provided from the released packet, which is a subset of the SNAIL examples.

When loading the training dataset, we converted the training schema links into a list of bullet points where each bullet point is . Then, we serialized the schema links with the question as the user prompt:

```
Question: <Question>
Schemas:
- <Table1>: <Column1>, <Column2>, ...
- <Table2>: <Column1>, <Column2>, ...
...
```

Therefore, for each training example, it looks like the following:

```
{
   "prompt": [
      {"role": "system", "content": SYSTEM_PROMPT}
      {"role": "user", "content": USER_PROMPT}
   ],
   "completion": [
      {"role": "assistant", "content": COMPLETION_TEXT}
   ],
}
```

where the completion text is the schema links as JSON object.

2 Final Pipeline Architecture

For the base model, we use `Qwen/Qwen2.5-1.5B-Instruct`, which has has less than 2B parameters. Based on our experiments, it gives us the best results compared to other models.

For LoRA adapter configuration, we mainly focused on rank, target modules, learning rate, and number of training epochs.

The following set of configurations is what we used in our final model:

- rank `r`: 64 (`lora_rank`: 128)
- target modules:
  - query projection `q_proj`

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

  - key projection `k_proj`
  - value projection `v_proj`
  - output projection of multi-head attention `o_proj`
  - MLP gate projection `gate_proj`
  - MLP up projection `up_proj`
  - MLP down projection `down_proj`
- learning rate: 1e-5
- number of epochs: 2

The prompt we used is the following:

> """You are a schema-linking model.
>
> Given a database schema and a natural-language question, output only valid JSON. The JSON must map table names to lists of referenced column names.
>
> Include tables even when no specific columns are referenced, using an empty list. Do not include explanations, markdown, or extra text.
>
> Example output format: "TableName1": ["Col1", "Col2"], "TableName2": ["Col3"]
>
> If a table is referenced but no specific columns are needed, map it to an empty list []."""

For post processing, we not only tried to decode the output and turn it into JSON, but also explicitly told the model to output an JSON object only. After the JSON object is extracted, we let it be the answer to the question.

3 Experimentation Methodology

Our experiments mainly focus on three aspects, base model, rank, and target modules.

3.1 Base Model

For base model, we experimented with the following models,

- `TinyLlama/TinyLlama-1.1B-Chat-v1.0`
- `HuggingFaceTB/SmolLM2-1.7B-Instruct`
- `Qwen/Qwen2.5-1.5B-Instruct`
- `Qwen/Qwen3-1.7B`

Ideally, the larger the model is, the better performance they can achieve. Therefore, we treated a relatively small model `TinyLlama/TinyLlama-1.1B-Chat-v1.0` as the baseline model and compare it with other slightly larger models.

Additionally, we also experimented with a non-instruct model `Qwen/Qwen3-1.7B` with slightly more parameters. Although it is not an instruct model that is designed to act as an assistant, we included it in our experiment to see if a larger model can still be competitive compared to instruct models.

3.2 Rank

We experimented `Qwen/Qwen2.5-1.5B-Instruct` with multiple different ranks. Ideally, larger ranks should have a better performance since more trainable parameters are injected into the model. However, training would take longer and more parameters need to be saved.

3.3 Target Modules

We started with only fine tune on key, query, and value projections as a baseline. Then, we added other linear layer parameters such as output projection of multi-head attention and gate, up, and down projections of the MLP. In addition, we also tried to see if fine tuning `wte` and `lm_head`, the

word token embedding and language modeling head to see if the fine-tuned embeddings can possibly improve the performance by capturing specific keywords or commonly seen vocabularies specific to this dataset.

3.4 Meaningfully Dinstinct Configurations

	Base Model	Rank	Target Modules
1	`TinyLlama-1.1B-Chat-v1.0`	32	`q_proj`, `k_proj`, `v_proj`
2	`SmolLM2-1.7B-Instruct`	32	`q_proj`, `k_proj`, `v_proj`
3	`Qwen2.5-1.5B-Instruct`	32	`q_proj`, `k_proj`, `v_proj`
4	`Qwen2.5-1.5B-Instruct`	64	`q_proj`, `k_proj`, `v_proj`, `gate_proj`, `up_proj`, `down_proj`
5	`Qwen3-1.7B`	64	`q_proj`, `k_proj`, `v_proj`, `gate_proj`, `up_proj`, `down_proj`
6	`Qwen2.5-1.5B-Instruct`	64	`q_proj`, `k_proj`, `v_proj`
7	`Qwen2.5-1.5B-Instruct`	128	`q_proj`, `k_proj`, `v_proj`
8	`Qwen2.5-1.5B-Instruct`	256	`q_proj`, `k_proj`, `v_proj`
9	`Qwen2.5-1.5B-Instruct`	512	`q_proj`, `k_proj`, `v_proj`

Table 1: Meaningfully distinct configurations

Configuration 1, 2, 3, 4, and 5 are used to compare different base models. Configuration 3, 6, 7, 8, and 9 are used to compare different ranks. Configuration 4 and 6 to differentiate target modules.

4 Results and Analysis

4.1 Base Model Experiments

Figure 1 shows the comparison between the three baseline models when using a rank of 32 and the same set of target modules. Across all evaluation metrics, `Qwen/Qwen2.5-1.5B-Instruct` consistently achieves the highest scores compared to `TinyLlama/TinyLlama-1.1B-Chat-v1.0` and `HuggingFaceTB/SmolLM2-1.7B-Instruct`. The improvement is particularly noticeable for table precision, table recall, and table F1, which shows that the Qwen model is more effective at capturing structured information from tables. While `SmolLM2-1.7B-Instruct` contains more parameters than `Qwen2.5-1.5B-Instruct`, its performance remains considerably lower, indicating that model size alone is not the primary factor determining effectiveness on this task. Instead, differences in pretraining data, instruction tuning, and model architecture likely also play a more significant role.

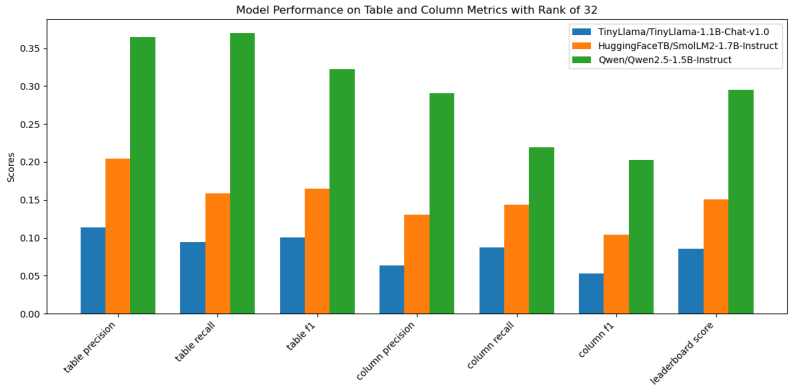


Figure 1: Base model comparison with rank of 32

Figure 2 compares the instruction-tuned `Qwen/Qwen2.5-1.5B-Instruct` model with the non-instruction-tuned `Qwen/Qwen3-1.7B` model under a rank of 64. The instruction-tuned model consistently outperforms the larger non-instruction model across every metric. This result suggests that

instruction tuning could provide substantial benefits for structured information extraction tasks by improving the model's ability to follow task-specific prompts and generate more consistent outputs. Furthermore, the fact that the smaller instruction-tuned model surpasses the larger non-instruction model highlights the importance of alignment and task-oriented fine-tuning.

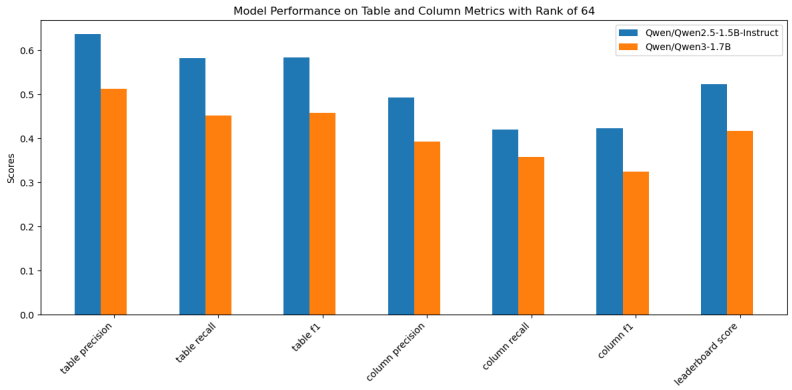


Figure 2: Instruct and non-instruct model compasion with rank of 64

4.2 Rank Experiment

For rank, we experimented with multiple values of rank. As shown in Figure 3, in our experiments, the larger the rank, the better performance it achieves as expected. However, we noticed that the performance improvement does not keep growing after some point unless it really gets significantly larger. In addition, with larger rank, the training time and checkpoint size gets a lot larger, so there is a tradeoff between efficiency and performance where fine tuning could be a lot longer and the model size could be a lot larger with only little performance improvement.

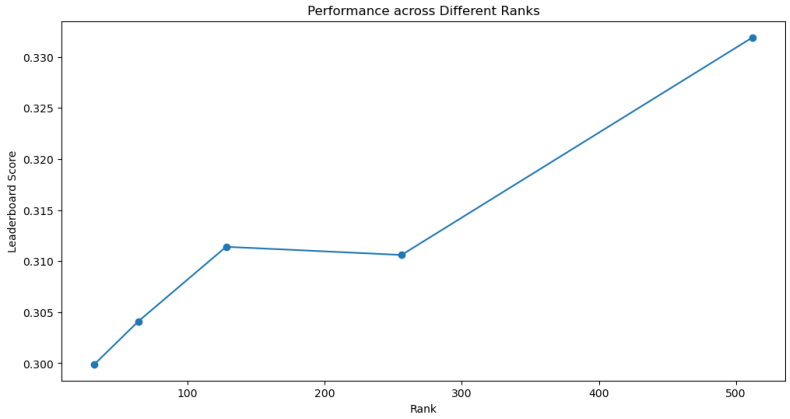


Figure 3: Rank comparison with `Qwen2.5-1.5B-Instruct`

4.3 Target Module Experiments

As shown in Figure 4, only training the attention projection layers (`q_proj`, `v_proj`, and `k_proj` yields the weakest performance across all metrics. Expanding LoRA adaptation to include the MLP layers (`gate_proj`, `up_proj`, and `down_proj`) results in a substantial improvement. This indicates that it requires not only modifications to attention mechanisms but also adaptations to the model's feedforward layers, which are responsible for storing and transforming task-specific knowledge. However, adding further modules (`wte` and `lm_head`) produces only marginal changes.

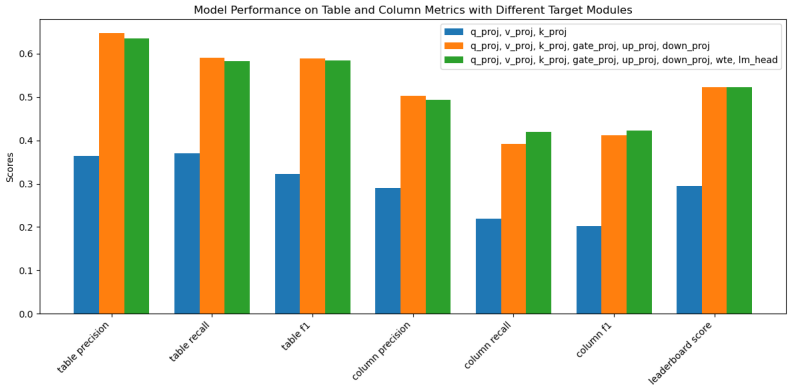


Figure 4: `Qwen2.5-1.5B-Instruct` with different target modules

5 AI Tools Statement

We used Claude to give us strategies to augment the training data. However, we did not end up using the augmented data.